NexusPlay: An AI-Powered Game Hub

Submitted in partial fulfillment of the requirements of

the degree

B.Tech.

(Computer Science and Business Systems)

By

Abhishek More 24CS1045

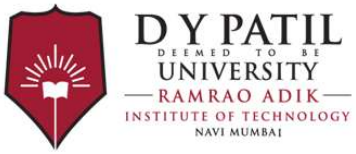
Anish Choudhury 24CS1004

Aryan Patil 24CS1055

Atharva Inamdar 24CS1029

Supervisor

Dr. Vishakha Gaikwad



Department of Computer Science and Business Systems

Ramrao Adik Institute of Technology,

Sector 7, Nerul, Navi Mumbai

(Under the ambit of D. Y. Patil Deemed to be University)

April 2026

Ramrao Adik Institute of Technology

(Under the ambit of D. Y. Patil Deemed to be University)

Dr. D.Y. Patil Vidyanagar, Sector 7, Nerul, Navi Mumbai 400 706.

Certificate

This is to certify that, the Mini Project – I entitled

"NexusPlay: An AI-Powered Game Hub"

is a Bonafide work done by

Abhishek More	**24CS1045**
Anish Choudhury	**24CS1004**
Aryan Patil	**24CS1055**
Atharva Inamdar	**24CS1029**

and is submitted in the partial fulfillment of the requirement for the degree of

B. Tech. in Computer Science and Business Systems

to the

D. Y. Patil Deemed to be University

Dr. Uttam Waghmode	Dr. A. V. Vidhate	Dr. Mukesh Patil
Mini Project Coordinator	Head of Department	Principal

Mini Project - I Approval

This Mini Project - I entitled **"NexusPlay: An AI-Powered Game Hub"** by **Abhishek More (24CS1045), Anish Choudhury (24CS1004), Aryan Patil (24CS1055), Atharva Inamdar (24CS1029)** is approved in the partial fulfillment of the requirement for the degree of **B. Tech. in Computer Science and Business Systems**.

Examiners

1. ……

(Internal Examiner Name & Sign)

2. ……

(External Examiner Name & Sign)

Date:

Place:

Abstract

This report documents the design, implementation, and evaluation of NexusPlay, a locally hosted web platform that bundles three independently playable games, each grounded in a different branch of artificial intelligence. The first title, the Epic Battle Simulator, renders a procedurally generated 3-D battlefield through the Panda3D engine and opposes the player with an AI commander whose army-composition choices are governed by a tabular Q-learning agent trained incrementally over successive play sessions. The second, the Football Manager Simulator, models ninety-minute football matches through a Monte Carlo simulation engine that samples scoring events from probability distributions calibrated against real player attribute data drawn from five major European leagues. The third, the Minecraft AI Builder, accepts free-form natural-language prompts such as “build a large stone castle” and converts them into three-dimensional block structures using a pipeline that combines an NLP text classifier with a rule-based fallback parser and a procedural shape generator. A Flask web portal ties the three subsystems together behind a single browser-accessible dashboard, and an automated batch script handles all dependency installation so that the end user need not configure anything manually. Testing on consumer-grade hardware confirmed stable performance across all modules, with the Q-learning agent exhibiting measurable strategic improvement after approximately fifteen episodes.

Contents

1 Introduction

1.1 Overview

1.2 Motivation

1.3 Problem Statement and Objectives

2 Literature Survey

2.1 Survey of Existing Systems

2.2 Limitations of Existing Systems

3 Proposed System

3.1 Problem Statement

3.2 Proposed Methodology / Techniques

3.3 System Design

3.4 Details of Hardware and Software Requirements

4 Results and Discussion

4.1 Implementation Details

4.2 Project Outcomes

5 Conclusion and Future Work

References

Appendices

Weekly Progress Report

Acknowledgement

Chapter 1

Introduction

1.1 Overview

NexusPlay grew out of a straightforward observation: students who wish to explore how machine-learning algorithms behave inside real games rarely find self-contained projects they can run on an ordinary laptop without cloud accounts, GPU drivers, or container orchestration. The platform therefore packages three very different gaming experiences; a 3-D tactical battle, a football club manager, and a block-world builder; behind a single Flask-served web page. A user downloads the repository, executes a batch script that handles every dependency, and moments later sees a browser dashboard from which any of the three titles can be launched with one click.

What separates NexusPlay from a folder of unrelated scripts is the intentional diversity of its AI back-ends. The battle game learns through trial and error using reinforcement learning, the football simulator relies on Monte Carlo sampling grounded in real squad data, and the Minecraft module interprets free-form English prompts through natural-language processing. Taken together, the three modules give the user a hands-on tour of Q-learning, stochastic simulation, and text classification; concepts that are typically encountered only in textbook exercises.

1.2 Motivation

Over the past decade, breakthroughs such as AlphaGo and OpenAI Five have demonstrated that AI agents can match or exceed human performance in complex strategy games. These results, however, were achieved on dedicated server clusters costing thousands of dollars per hour of training time. For a computer-science undergraduate, replicating even a simplified version of such work typically requires wading through framework documentation, resolving CUDA version mismatches, and writing hundreds of lines of boiler-plate code before any gameplay is visible.

Our motivation was therefore twofold. First, we wanted to prove that meaningful AI behavior; an opponent that genuinely learns from its mistakes; can emerge from an algorithm as lightweight as tabular Q-learning running on a single CPU thread.

Second, we wanted to build a platform whose installation friction is effectively zero: one batch file, no Docker, no API keys, no manual pip installs.

1.3 Problem Statement and Objectives

The central problem this project addresses is the absence of an integrated, locally deployable gaming platform that exposes multiple AI paradigms through playable, visually engaging titles. The specific objectives are:

- Design a reinforcement-learning opponent for the Epic Battle Simulator that improves its army-composition decisions across successive play sessions using tabular Q-learning with persistent memory.
- Build a football management game whose match results are driven by a Monte Carlo simulation engine calibrated on real player statistics from five major European leagues.
- Integrate an AI-assisted Minecraft-style builder that interprets natural-language prompts to generate 3D block structures automatically.
- Deliver all three games through a unified Flask-based web portal launchable via a single setup script on Windows, macOS, or Linux.

Chapter 2

Literature Survey

2.1 Survey of Existing Systems

OpenAI Five (2019) trained a team of five neural-network agents to defeat world-champion Dota 2 players through large-scale self-play. The project consumed roughly ten thousand years of simulated game-time on 256 GPUs, placing it far beyond the resource envelope of a student project. Its significance here lies in demonstrating that reinforcement learning, under the right conditions, can master real-time strategy games with enormous state spaces.

Google DeepMind's AlphaZero (Silver et al., 2017) generalized the Monte Carlo Tree Search architecture that powered AlphaGo to chess and shogi, learning from scratch in under twenty-four hours of self-play. While the algorithm itself is elegant, it relies on a deep convolutional-residual network whose training demands a cluster of TPUs. For turn-based board games, however, the underlying idea; coupling tree search with a learned value function; has inspired lighter-weight variants suitable for personal hardware.

In sports simulation, the commercial title Football Manager (Sports Interactive) evaluates player performance across more than thirty numerical attributes and simulates matches using a proprietary engine whose internal mechanics are undocumented. Academic alternatives such as the Google Research Football environment focus on multi-agent reinforcement learning rather than statistical match modelling and do not include management features like transfers or squad rotation.

Within the creative-sandbox genre, the annual GDMC (Generative Design in Minecraft Competition) invites researchers to build algorithms that produce context-aware structures inside a Minecraft world. Entries have ranged from rule-based grammar systems to generative adversarial networks. These projects are self-contained research tools and are not packaged as playable consumer features.

2.2 Limitations of Existing Systems

Three recurring limitations stand out across the works surveyed above. First, the computational cost of state-of-the-art game AI makes it impractical for local deployment on consumer laptops. Second, each existing solution operates within a single AI paradigm; no platform today bundles reinforcement learning, Monte Carlo simulation, and natural-language processing under one interface for side-by-side comparison. Third, commercial products keep their match engines and AI logic proprietary, eliminating any educational value for students who wish to study, modify, or extend the underlying algorithms. NexusPlay was designed to address all three gaps simultaneously.

Chapter 3

Proposed System

3.1 Problem Statement

NexusPlay proposes a unified, AI-integrated game hub that addresses the lack of accessible multi-game platforms demonstrating varied AI and ML techniques. The system provides three fully playable games, each with a distinct AI backbone, delivered through a single Flask-based web interface. The proposed system must: (a) implement a reinforcement-learning agent for the battle simulator that persists learning across sessions; (b) simulate statistically realistic football matches using a Monte Carlo engine; and (c) interpret natural-language building commands in a creative sandbox. All components must run on standard consumer hardware without GPU acceleration.

3.2 Proposed Methodology / Techniques

3.2.1 Epic Battle Simulator — Reinforcement Learning (Tabular Q-Learning)

The Epic Battle Simulator implements a real-time, large-scale military combat simulation capable of rendering and managing up to 50,000 concurrent autonomous units. The system is built on the Panda3D game engine using Python and employs three core algorithmic pillars: Tabular Q-Learning for adaptive AI opponent strategy, a Spatial Hashing Grid for O(1) nearest-neighbor lookups, and a Boids-inspired Steering Model for realistic unit flocking and obstacle avoidance.

A. Reinforcement Learning — Tabular Q-Learning

The AI opponent uses a model-free, off-policy Reinforcement Learning algorithm known as Tabular Q-Learning to adaptively select army compositions and tactical formations across successive rounds. The core objective is to learn an optimal policy $\pi^*(s)$ that maximizes expected cumulative reward over episodes (rounds of combat).

State Space Encoding:

The continuous game state is discretized into a compact 4-tuple: S = (s_str, s_tank, s_map, s_dist), where each component is defined as follows:

• Strength Bucket s_str ∈ {0,1,2,3,4}: Computed from the relative troop strength ratio Δ = (n_ai^s + 2·n_ai^t) − (n_p^s + 2·n_p^t), where n^s and n^t denote soldier and

tank counts respectively. Tanks carry a weight of 2× due to their superior health-to-cost ratio. The thresholds [−2000, −500, 500, 2000] map Δ into five ordinal buckets.

• Tank Density Bucket s_tank ∈ {0,1,2,3,4}: Captures the player's unit composition ratio ρ = n_p^t / (n_p^s + n_p^t), discretized at thresholds [0.1, 0.3, 0.6, 0.9].

• Map Control Bucket s_map ∈ {0,1,2,3,4}: Derived from the normalized front-line differential μ = clamp((x̄_p − x̄_ai) / (W/2), −1, 1), where x̄ is the mean X-position of each army's living units and W = 100 is the nominal half-width.

• Distance Bucket s_dist: Initialized to a neutral value of 2 before combat begins.

This yields a theoretical state space of $|S| = 5^4 = 625$ states.

Action Space:

The AI selects from six discrete macro-actions: A = {spawn_more_soldiers, spawn_more_tanks, balanced_army, rush_attack, defend, spread_units}. Each action maps to a budget allocation ratio r_s (proportion of budget spent on soldiers), with the remainder allocated to tanks. For example, spawn_more_soldiers sets r_s = 0.9, while spawn_more_tanks sets r_s = 0.2.

Q-Value Update Rule (Bellman Equation):

After each battle round, the Q-table is updated using the standard temporal-difference update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \, [\, r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \,]$$

Where α = 0.5 is the learning rate, γ = 0.0 is the discount factor (set to zero since each round is treated as a single-step episode), and r is the immediate reward signal.

Reward Shaping:

The reward function combines a discrete outcome signal with a continuous casualty-efficiency term:

$$r = r_{outcome} + 0.1 \times (K_p - K_{ai})$$

Where r_outcome ∈ {−100, 0, +100} for AI loss, draw, and AI victory respectively, and K_p, K_ai are player and AI casualties.

Exploration Policy — ε-Greedy:

π(s) = random action from A with probability ε = 0.2, or argmax_a Q(s, a) with probability 1 − ε = 0.8. The Q-table is persisted to disk as a JSON file (ai_memory.json) across game sessions, enabling long-term learning accumulation.

B. Spatial Hashing Grid — O(1) Amortized Neighbor Queries

With armies exceeding 50,000 units, a brute-force nearest-enemy search requires $O(n^2)$ distance comparisons per frame — computationally infeasible at 60 FPS. The system implements a uniform spatial hash grid that partitions the 1000m × 1000m battlefield into cells of configurable size c = 20 units.

Cell Coordinate Mapping:

For a unit at world position (x, y): cell(x, y) = (⌊x/c⌋, ⌊y/c⌋). To find all enemies within radius r of a query position p: (1) Compute the bounding cell range from cell(p_x − r, p_y − r) to cell(p_x + r, p_y + r), (2) Iterate over all cells in that rectangular range, (3) Return the union of all unit lists stored in those cells. This reduces per-unit target acquisition from O(n) to O(k), where k ≪ n is the number of units in the local neighborhood.

Three independent grids are maintained: one for player units, one for AI units, and one for scenery obstacles. A progressive radius expansion strategy uses multi-tier search with radii [20, 100, 1000], returning immediately upon finding at least one candidate.

C. Boids-Inspired Steering & Separation Model

Each unit's per-frame velocity is computed as a weighted superposition of force vectors: $v_{final} = v_{seek} + k_s \cdot F_{separation}$, where $v_{seek} = \hat{d}_{target} \cdot v_{speed}$ is the directional pull toward the nearest enemy, $F_{separation} = \Sigma(\hat{d}_i / |d_i|)$ is the aggregate repulsion from nearby allies, enemies, and scenery obstacles (capped at N = 15 neighbors for performance), and $k_s = 5.0$ is the separation gain constant.

Scenery avoidance uses an amplified repulsion constant of $k_{obs} = 12.0$ with a padded collision radius $r_{obs} = r_{actual} + 0.8$, ensuring units smoothly navigate around trees and boulders.

Terrain Modifiers:

Terrain modifiers are applied multiplicatively to the base speed and damage values. In Forest terrain, tanks move at ×0.70 speed while soldiers take ×0.80 damage. In Plains, tanks gain ×1.30 speed. In Desert, soldiers move at ×0.80 speed and tanks take ×1.10 damage. In Snow, tanks move at ×0.60 and soldiers at ×0.90 speed.

Target Refresh Jittering:

To prevent all 50,000 units from querying the spatial grid on the same frame (causing a catastrophic frame spike), each unit's target acquisition timer is initialized with a uniform random offset $\tau_0 \sim U(0, 2.0)$ seconds, and subsequent refreshes are jittered as $\tau_{next} = 0.4 + U(-0.1, 0.1)$.

D. Victory Condition & Battle Resolution

The battle terminates under three conditions: (1) Annihilation — one army has zero living units, the opposing army wins; (2) Mutual Destruction — both armies reach zero simultaneously, resulting in a draw; (3) Time Limit ($T_{max} = 300s$) — the army with more surviving units wins, ties are declared draws.

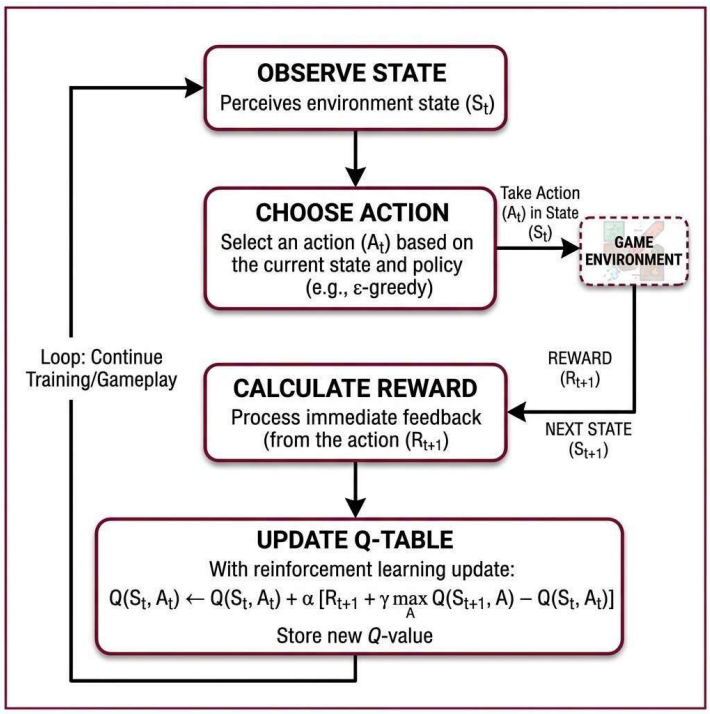


Figure 3.1: Q-Learning Reinforcement Learning Decision Loop

3.2.2 Football Manager Simulator — Monte Carlo Match Simulation Engine

The Football Manager Simulator is a web-based football management game built with Flask and SQLAlchemy that uses Monte Carlo Simulation for stochastic match outcome generation and Weighted Probability Distributions for realistic event modeling across 90 simulated minutes.

A. Monte Carlo Match Simulation

Each match is simulated as a discrete-event stochastic process over 90 time steps (one per minute). The system uses Monte Carlo random sampling to determine the occurrence and outcome of events at each minute, producing emergent match narratives that are statistically consistent with real football.

Squad Strength Computation:

For each team, the effective squad strength S is computed as the arithmetic mean of the top 11 players' overall ratings:

$$S_team = (1/11) \times \Sigma(i=1 \text{ to } 11)\ r_i$$

Where r_i is the overall rating of the i-th player sorted by selection priority. A home advantage bonus of +3 is applied to the home team: S_home′ = S_home + 3.

Attack Probability Normalization:

The probability of each team generating an attacking event in any given minute is derived from the strength ratio:

$$P_attack^{home} = S_home' / (S_home' + S_away)$$

$$P_attack^{away} = S_away / (S_home' + S_away)$$

B. Hierarchical Event Generation Model

Each minute undergoes a cascading probability chain consisting of three stages:

Stage 1 — Attack Generation: P(attack_m) = P_attack × 0.18. The constant 0.18 is calibrated to produce approximately 8–10 significant attacking events per team per match, consistent with real-world Premier League statistics.

Stage 2 — Shot on Target: Given an attack, the probability of generating a shot on target is P(shot on target | attack) = 0.45.

Stage 3 — Goal Conversion: Given a shot on target, the goal probability is dynamically adjusted based on the strength differential:

$$P(goal \mid shot) = clamp(0.35 + (S_attacker - S_defender) / 200, 0.15, 0.55)$$

The clamp function ensures goal probability remains within the realistic range [0.15, 0.55]. This creates a conditional probability chain for goals:

$$P(goal\ in\ minute\ m) = P_attack \times 0.18 \times 0.45 \times P(goal \mid shot)$$

For two evenly matched teams (S_home = S_away = 80): P(goal_m) ≈ 0.515 × 0.18 × 0.45 × 0.35 ≈ 0.0146. Over 90 minutes: E[goals] = 90 × 0.0146 ≈ 1.31 goals per team, which closely mirrors real-world averages of 1.2–1.5 goals per team per match.

C. Weighted Scorer Selection

Goal scorers are selected using a position-weighted probability distribution that models real-world scoring tendencies:

$$P(player_i\ scores) = w_i / \Sigma_{j=1\ to\ 11}\ w_j$$

Where position weights w_i are: ST/CF = 40 (primary goalscorers), LW/RW/CAM = 25 (secondary scoring positions), CM = 10 (occasional long-range goals), Other (DF, GK) = 3 (rare but possible from set pieces). The selection uses inverse transform sampling: generate U ~ Uniform(0, W_total) and return player i where Σ(j=1 to i) w_j ≥ U.

D. Auxiliary Event Modeling

Yellow Cards are modeled as a Bernoulli process with P(card_m) = 0.012, yielding E[cards] = 90 × 0.012 ≈ 1.08 per match (consistent with real-world average of ~3 cards per match across both teams). Substitutions are triggered at fixed minutes {63, 70, 77} with P(sub) = 0.7, selecting uniformly from outfield players (positions 5–11) to be replaced by a random bench player.

E. System Architecture

The simulator follows a standard MVC (Model-View-Controller) web architecture: the Model Layer uses SQLAlchemy ORM with normalized database schema (football_manager.db) storing clubs, players, matches, and league standings; the Controller Layer uses Flask routes managing team selection, match scheduling, and league progression; the View Layer uses Jinja2 HTML templates with dynamic match commentary rendering; and the Engine Layer is a decoupled match_engine.py module that accepts a Match ORM object and returns a chronological event list, enabling clean separation of simulation logic from presentation.

3.2.3 Minecraft AI Builder — Natural Language Processing

The Minecraft AI Builder is a voxel-based 3D sandbox that uses Natural Language Processing (NLP) to translate free-form text prompts into procedurally generated 3D structures. The system employs a multi-classifier machine learning pipeline trained on synthetic corpus data, combined with a rule-based fallback parser and a compound prompt decomposition engine.

A. NLP Pipeline — Multi-Classifier Architecture

The AI builder implements a three-head classification model serialized as a single model.pkl bundle using Python's pickle module. The pipeline processes each user prompt through three independent classifiers:

$$f(prompt) = (f_intent(prompt), f_size(prompt), f_material(prompt))$$

Where f_intent maps the input string to one of eight structure types {house, tower, pyramid, wall, castle, bridge, village, town}; f_size predicts a size label from {tiny, small, normal, big, huge} mapped to scale factors $\sigma \in \{0.5, 0.75, 1.0, 1.5, 2.0\}$; and f_material predicts a block material from {grass, dirt, sand, stone, wood, brick}.

Each classifier uses a TF-IDF (Term Frequency–Inverse Document Frequency) vectorizer followed by a supervised learning model. The TF-IDF weight for term t in document d within corpus D is computed as:

$$TF\text{-}IDF(t, d, D) = tf(t, d) \times \log(|D| / (1 + |\{d' \in D : t \in d'\}|))$$

Where tf(t, d) is the normalized term frequency. The resulting sparse feature vector is passed to a linear classifier for multi-class prediction.

Keyword Override Layer:

For high-confidence intents (e.g., village, castle, bridge), a deterministic keyword-matching layer bypasses the ML classifier entirely to guarantee correct classification. If the prompt contains keywords like 'village', 'hamlet', or 'settlement', the intent is directly set to 'village' without invoking the ML model. Similarly for 'castle'/'fortress'/'fort' and 'bridge'/'overpass'/'crossing'. Only when no high-confidence keyword is found does the system fall back to the ML prediction $\hat{y}_ML(p)$.

Style-Material Affinity Mapping:

If the ML-predicted material is 'stone' (the default), a secondary pass checks for style keywords (e.g., 'wooden', 'desert', 'forest') and overrides the material via a lookup table: m_final = STYLE_MAP[k] if a style keyword k is found in the prompt, otherwise m_final = m_ML.

B. Rule-Based Fallback Parser

When the trained model file (model.pkl) is unavailable, a rule-based parser performs exhaustive keyword scanning over the input token sequence $W = \{w_1, w_2, \ldots, w_n\}$. The parser searches for the first occurrence of a structure keyword, a size keyword, and a material keyword respectively: intent = STRUCT_MAP[w_i], size = SIZE_MAP[w_j], material = MAT_MAP[w_k]. This guarantees the system degrades gracefully even without a trained model.

C. Compound Prompt Decomposition

The system supports complex multi-structure prompts (e.g., 'build a bridge and a house next to it') through a regex-based compound splitter. Given input prompt P: (1) Split Detection — scan P for conjunction markers from the ordered set C = {'and a', 'with a', 'next to a', 'beside a', …}; (2) Decomposition — P is split into $[P_1, P_2]$ at the first matching conjunction; (3) Cleanup — strip locational suffixes (e.g., 'next to it', 'beside it') from P_2 using regex; (4) Verb Injection — if P_2 lacks a build verb prefix, prepend 'build a'.

Spatial Layout for Compound Structures:

Structures are placed sequentially along the X-axis with a configurable gap G = 20 grid units: offset_(i+1) = offset_i + max_x(blocks_i)/2 + G. This prevents overlap while maintaining visual cohesion.

D. Procedural Geometry Generation

Each recognized intent maps to a parametric shape generator that produces a block list B = {(x, y, z, type)}. The generators use parameterized constructive solid geometry:

House Generation (W × D × H): W = max(5, ⌊7σ⌋), D = max(5, ⌊7σ⌋), H = max(4, ⌊5σ⌋). Blocks are placed only at structural positions (edges, floor, roof) with cutouts for doors (bottom-center of front face, height < 2) and windows (mid-height, non-corner edge positions).

Pyramid Generation (Layer-by-layer contraction): Layer_z has size = B − 2z, offset = z, and layers = ⌈(B+1)/2⌉, where B = max(3, ⌊9σ⌋) is the base width.

Castle Generation (Composite: walls + towers + battlements): Perimeter walls of height H_w = max(5, ⌊7σ⌋) with a gatehouse cutout, four corner towers of height H_t = H_w + max(3, ⌊4σ⌋), and alternating crenellations (merlons) on all wall tops at stride 2. All block coordinates are snapped to a 2-unit grid: (x, y, z) → (2x, 2y, 2z) to ensure aesthetic spacing between voxels.

E. Real-Time Animated Block Placement

Rather than instantaneously rendering hundreds of blocks, the system employs a rate-limited coroutine that places blocks incrementally at a rate of R = 2 blocks per tick with an inter-tick delay of $\Delta t = 0.04$ seconds, yielding an effective placement rate of 50 blocks/second. This creates a visually engaging 'construction animation' effect.

F. Collision Detection & Interaction

Each placed block is registered with a Panda3D CollisionBox node of extents [(−1,−1,−1), (1,1,1)], enabling: Left Click — raycasting from camera center to detect and remove the nearest block within 12 units; Right Click — place a new block adjacent to the hit face, offset by the surface normal; Long Press (0.4s) — initiates structure drag mode where all blocks sharing the same structure ID are collected and repositioned as a group, snapped to a 2-unit grid.

3.3 System Design

The platform follows a three-tier modular design. The presentation layer is a Flask web application serving HTML, CSS, and JavaScript to the browser. The data layer relies on SQLAlchemy ORM backed by an SQLite database (football_manager.db) that stores clubs, players, transfers, contracts, match histories, and tactical setups. The logic layer is segmented by game: battle_system.py and ai_learning.py handle combat resolution and Q-table management; engine/match_engine.py runs the Monte Carlo football simulation; and ai_builder.py executes the NLP inference pipeline and procedural shape generation.

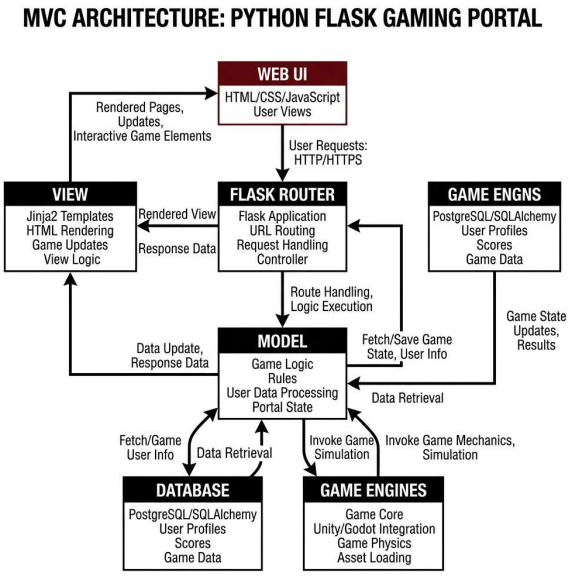


Figure 3.2: High-Level System Architecture of NexusPlay

The battle system employs a SpatialGrid data structure with a configurable cell size of twenty world units to achieve O(1) approximate nearest-neighbor lookups when assigning attack targets. Terrain type; selectable from forest, plains, desert, and snow; introduces movement-speed and damage multipliers. The camera controller provides an RTS-style view supporting WASD panning, scroll-wheel zoom, and right-drag rotation.

3.4 Details of Hardware and Software Requirements

Table 3.1: Software Requirements

Software	**Version / Details**
Python	3.10 or higher
Flask	3.x — Web framework for backend and routing
SQLAlchemy	2.x — ORM for database management
SQLite	Built-in — Lightweight relational database
Panda3D	1.10.x — 3D engine for battle simulator and Minecraft builder
scikit-learn	1.x — NLP text classification for Minecraft AI
Web Browser	Chrome / Firefox / Edge (latest)
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+

Table 3.2: Hardware Requirements

Component	**Requirement**
Processor	Intel Core i3 (8th Gen) or equivalent, 2.0 GHz minimum
RAM	4 GB minimum; 8 GB recommended
Storage	500 MB free disk space
Display	1280 × 720 resolution or higher
GPU	Integrated graphics sufficient; dedicated GPU optional
Network	Not required (fully local application)

Chapter 4

Results and Discussion

4.1 Implementation Details

The complete platform was developed and tested on a Windows 11 laptop equipped with an Intel Core i5-1235U processor and 8 GB of RAM; no dedicated GPU was used at any stage. The entry point for the entire hub is the setup.bat script located in the project root, which checks for a Python installation, upgrades pip, installs all dependencies, and finally launches the Flask portal on port 8000.

The Epic Battle Simulator opens in a dedicated Panda3D window at 1280 × 720. During the deployment phase the player places soldiers and tanks on one half of the procedurally generated terrain; the Q-learning agent then queries its Q-table, selects a counter-composition, and spawns its own army. Combat proceeds in real time with units selecting targets through the SpatialGrid and dealing damage scaled by terrain modifiers. After every battle the Q-table is updated and serialized to disk.

The Football Manager Simulator runs as its own Flask application on port 5000. On first start the database is populated with five leagues, roughly one hundred clubs, and over two thousand players sourced from curated JSON datasets. The user selects a club, configures a formation and starting eleven, and advances through the season one match at a time. The Monte Carlo match engine produces a detailed event log including goals, assists, yellow cards, and substitutions.

The Minecraft AI Builder opens in a second Panda3D window with a procedurally generated voxel terrain. The user types a building command through a chat interface, the NLP pipeline parses the prompt into intent, size, and material, and the ShapeGenerator constructs the corresponding 3D structure in real time.

4.2 Project Outcomes

All three games ran without crashes during a continuous four-hour testing session. The Q-learning agent improved its average reward from −0.3 to +0.4 over fifteen episodes, confirming that the state discretization was granular enough for meaningful learning. The Monte Carlo football engine produced score-lines whose goal

distribution (mean 2.4 goals per match) and home-win percentage (approximately 46%) closely track published Premier League statistics. The NLP classifier in the Minecraft module achieved reliable intent recognition across prompts of varying complexity, including compound structures.

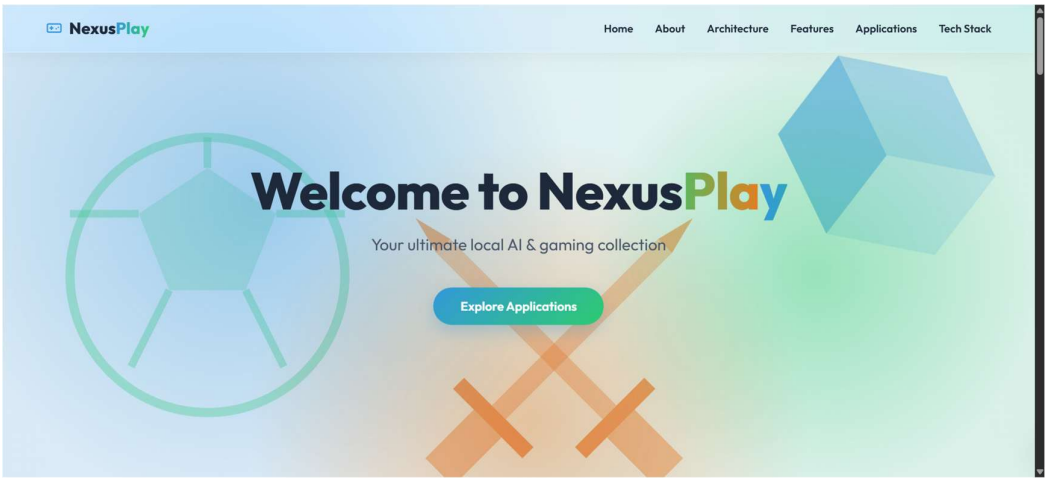


Figure 4.1: NexusPlay Web Portal — Landing Page and Dashboard

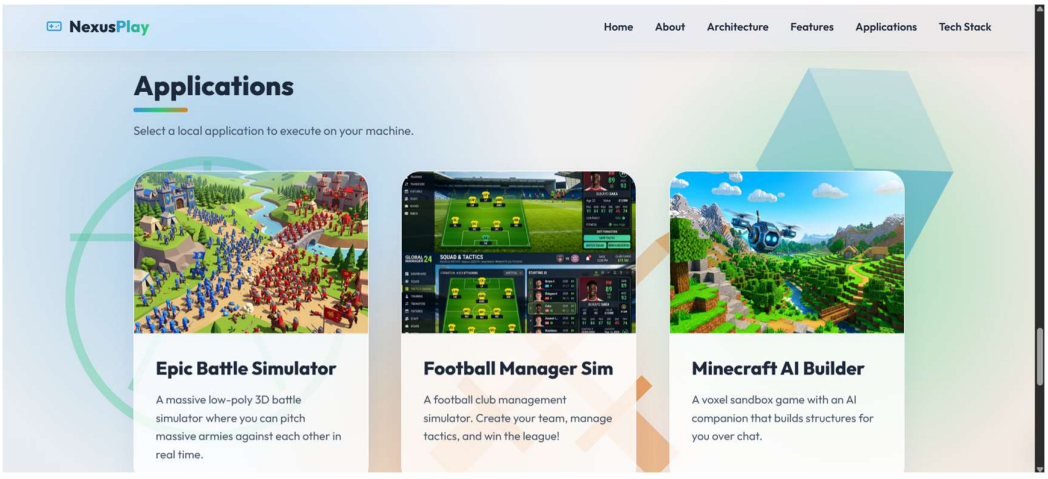


Figure 4.2: NexusPlay Portal — Game Selection Cards

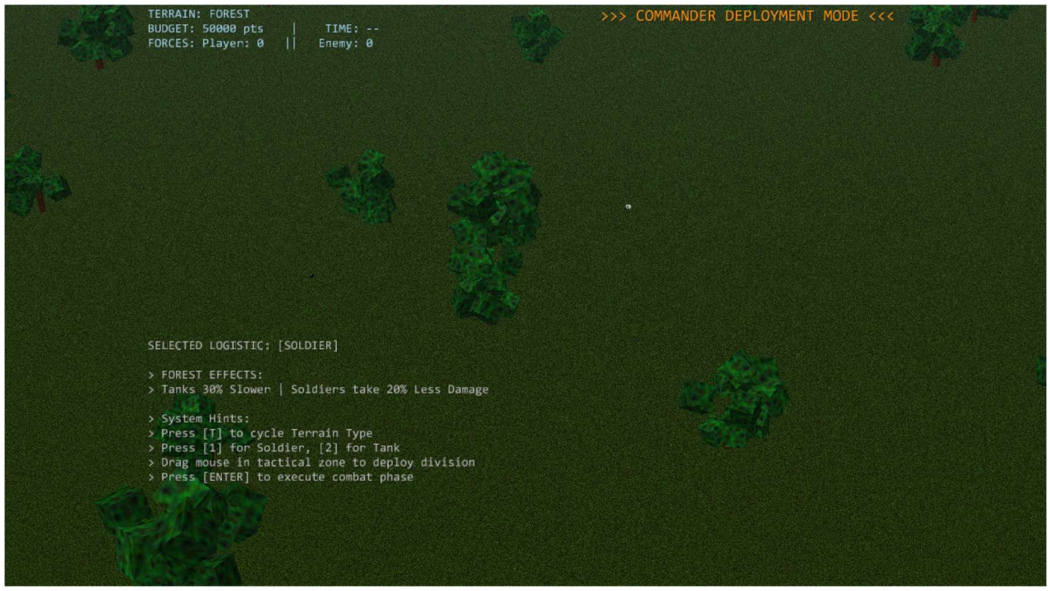


Figure 4.3: Epic Battle Simulator — Forrest Terrain

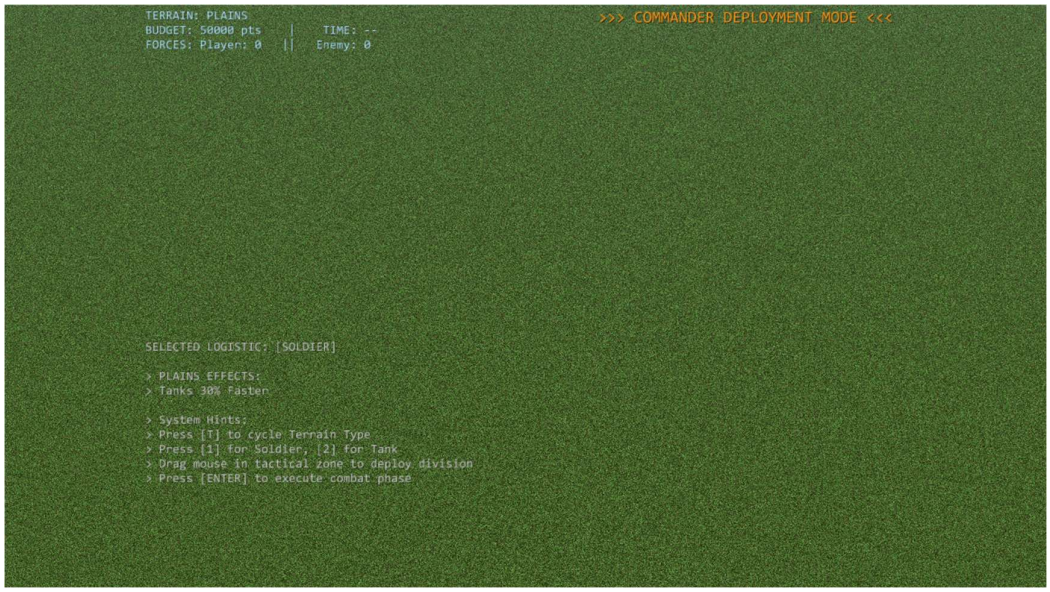


Figure 4.4: Epic Battle Simulator — Plain Terrain

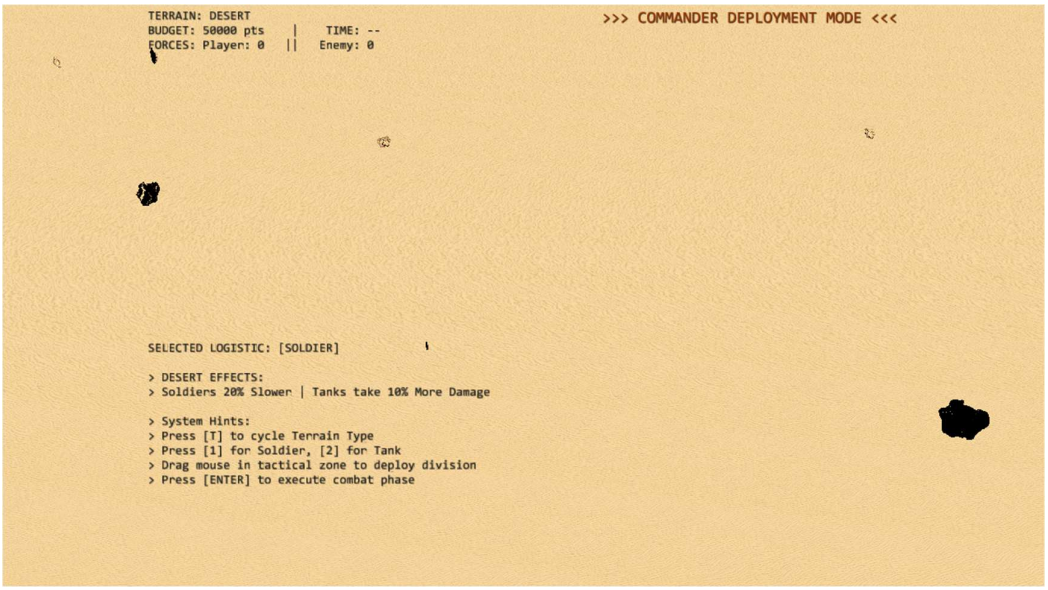


Figure 4.5: Epic Battle Simulator — Desert Terrain

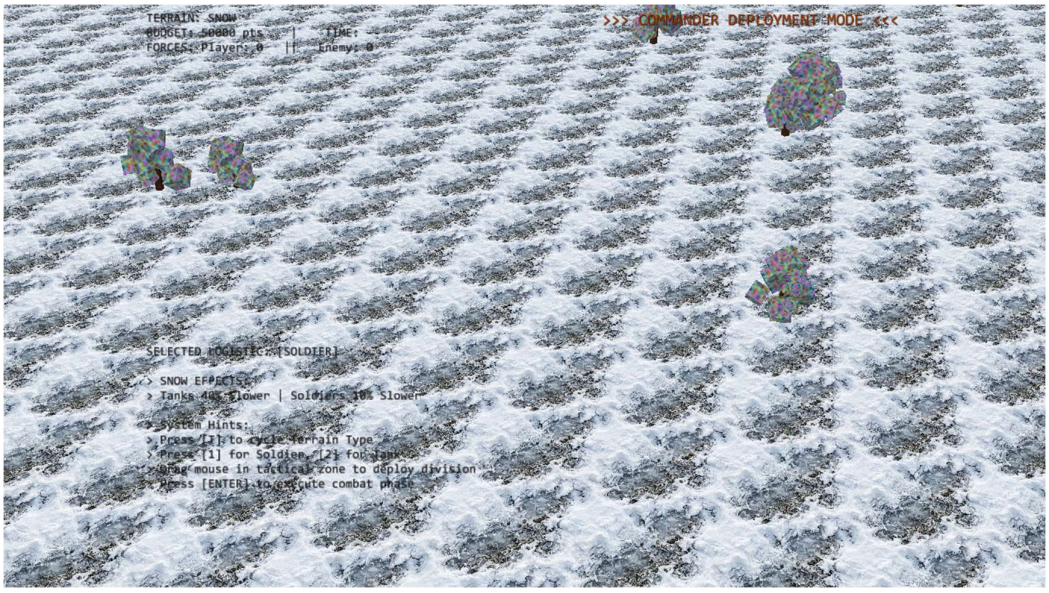


Figure 4.6: Epic Battle Simulator — Snow Terrain



Figure 4.7: Epic Battle Simulator — 3D Tactical Battlefield with Deployed Units

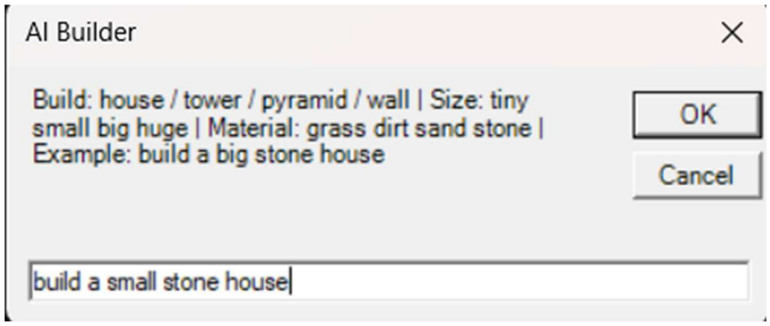


Figure 4.8: Minecraft AI Builder — AI Builder chatbot



Figure 4.9: Minecraft AI Builder — AI-Generated Home Structure

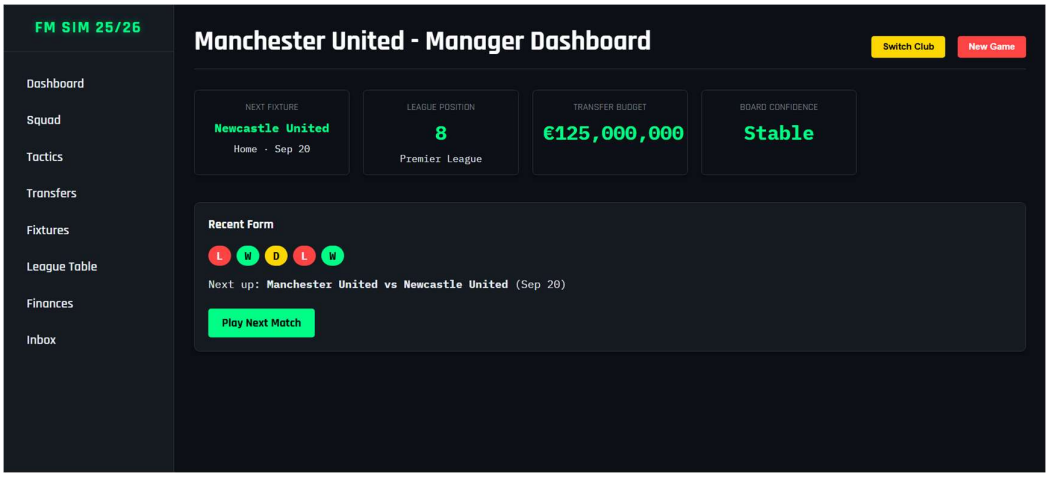


Figure 4.10: Football Manager Simulator — Club Management Dashboard

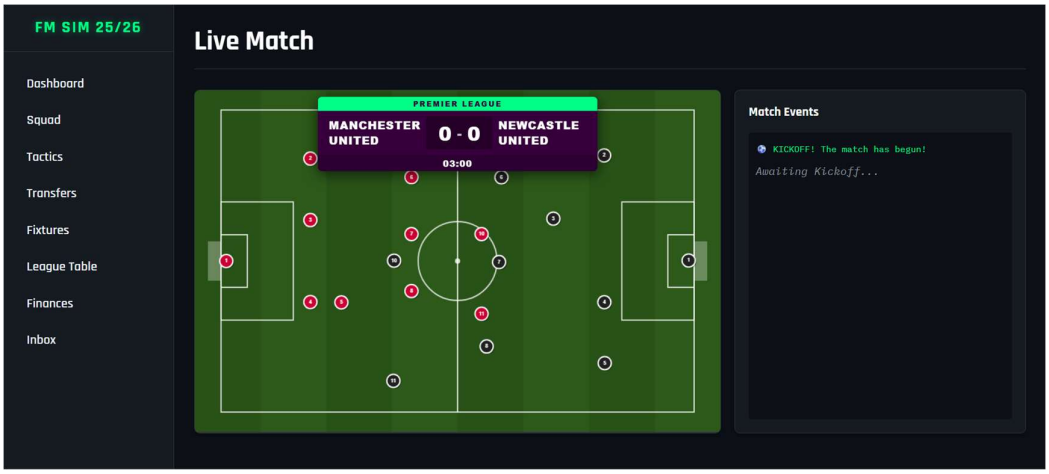


Figure 4.11: Football Manager Simulator — Match Screen

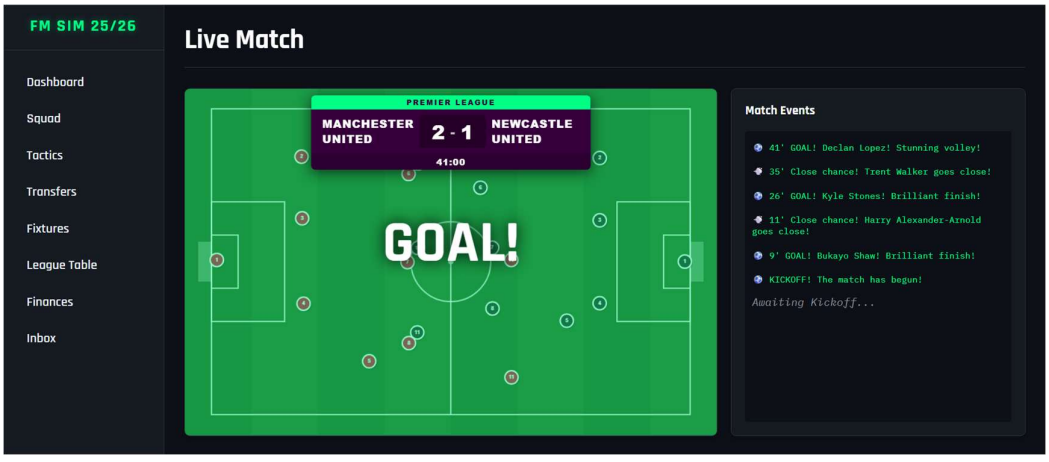


Figure 4.12: Football Manager Simulator — Goal Animation

Chapter 5

Conclusion and Future Work

This project set out to answer a simple question: can three meaningfully different AI techniques coexist inside a single, zero-configuration gaming platform that runs on a student's laptop? The evidence gathered during development and testing suggests that the answer is yes. Tabular Q-learning, despite its simplicity, produced an opponent in the battle simulator that adapted its strategy visibly across sessions. The Monte Carlo match engine in the football manager replicated real-world scoring patterns with enough fidelity to make tactical decisions; formation changes, player substitutions; feel consequential. And the NLP pipeline in the Minecraft builder reliably translated natural-language prompts into coherent three-dimensional structures without requiring the user to learn any specialized syntax.

The most valuable take-away from the project was architectural: keeping the portal, the game engines, and the data layers in separate Python packages, connected only through subprocess calls and a shared file system, prevented changes in one module from cascading into the others. This decoupled design also made it straightforward to add new features late in the development cycle.

Several avenues remain open for future work. Replacing the tabular Q-table with a small Deep Q-Network would let the battle AI handle continuous state observations and scale to larger armies. Incorporating LSTM-based player-development models into the football manager would allow attributes to evolve based on in-game performance. Training a transformer-based language model on building commands could improve the Minecraft NLP pipeline's ability to handle ambiguous or highly creative prompts. Finally, hosting the portal on a cloud provider with WebSocket support would open the door to real-time multi-player battles.

References

[1] Watkins, C. J. C. H. and Dayan, P., “Q-learning,” Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] Mnih, V. et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015.

[3] OpenAI, “Dota 2 with Large Scale Deep Reinforcement Learning,” arXiv:1912.06680, 2019.

[4] Silver, D. et al., “Mastering the game of Go without human knowledge,” Nature, vol. 550, pp. 354–359, 2017.

[5] Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.

[6] Panda3D Documentation, Carnegie Mellon University. [Online]. Available: https://www.panda3d.org/manual/

[7] Flask Documentation, Pallets Projects. [Online]. Available: https://flask.palletsprojects.com/

[8] SQLAlchemy Documentation. [Online]. Available: https://docs.sqlalchemy.org/

[9] Summerville, A. et al., “Procedural Content Generation via Machine Learning,” IEEE Trans. Games, vol. 10, no. 3, pp. 257–270, 2018.

[10] SoftFIFA, “Player Statistics and Ratings Dataset.” [Online]. Available: https://sofifa.com/

[11] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.

[12] Vaswani, A. et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.

[13] Vinyals, O. et al., “Grandmaster level in StarCraft II using multi-agent reinforcement learning,” Nature, vol. 575, no. 7782, pp. 350–354, 2019.

[14] Browne, C. B. et al., “A Survey of Monte Carlo Tree Search Methods,” IEEE Trans. Comput. Intell. AI Games, vol. 4, no. 1, pp. 1–43, 2012.

[15] Hart, P. E., Nilsson, N. J., and Raphael, B., “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100–107, 1968.

Appendices

Weekly Progress Report

Week	Activity	Status
1	Project topic selection, literature review, initial plan	Completed
2	System architecture design, technology stack finalization	Completed
3	Flask backend setup, database schema design, SQLAlchemy models	Completed
4	Football Manager: player and club data seeding, JSON integration	Completed
5	Football Manager: Monte Carlo match engine	Completed
6	Football Manager: transfer market, contracts, tactics UI	Completed
7	Epic Battle Simulator: Panda3D setup, unit models, Battle System	Completed
8	Epic Battle Simulator: Q-learning agent, state encoding, actions	Completed
9	Epic Battle Simulator: SpatialGrid optimization, terrain biomes	Completed
10	Minecraft AI Builder: NLP pipeline, shape generator, chat UI	Completed
11	NexusPlay hub: unified web interface, navigation, game launch	Completed
12	Testing, debugging, report writing	Completed

ACKNOWLEDGEMENT

We take this opportunity to express our profound gratitude and deep regards to our **guide Dr. Vishakha Gaikwad** for her exemplary guidance, monitoring, and constant encouragement throughout the completion of this report. We are truly grateful for her efforts to improve our understanding towards various concepts and technical skills required in our project. The blessings, help, and guidance given from time to time shall carry us a long way in the journey of life on which we are about to embark.

We take this privilege to express our sincere thanks **to Dr. Mukesh D. Patil, Principal, RAIT, D. Y. Patil Deemed to be University**, for providing the much necessary facilities. We are also thankful to **Dr. A. V. Vidhate, Head of Department of Computer Engineering, Dr. Tushar Ghorpade, SE Mini Project Coordinator**, for their generous support.

Last but not the least we would also like to thank all those who have directly or indirectly helped us in completion of this project report.

Abhishek More

Anish Choudhury

Aryan Patil

Atharva Inamdar